



Informe Inicial Proyecto Spring Boot

Miembros del Equipo

- Angela Lozano Pulido
- Jose David Menesses
- Gabriel Alfonso Vera

Nombre del Proyecto: MusicZone

Justificación y/o pertinencia de la temática

El consumo de música en plataformas digitales ha crecido significativamente en los últimos años, convirtiéndose en una de las principales formas de entretenimiento a nivel mundial. Aplicaciones como servicios de streaming han transformado la manera en que los usuarios organizan, buscan y gestionan su contenido musical.

El desarrollo de MusicZone permite simular el funcionamiento básico de una plataforma musical digital, enfocándose en la gestión de listas de reproducción y la consulta de un catálogo de canciones. Este proyecto resulta pertinente porque integra conceptos fundamentales del desarrollo de aplicaciones web modernas, tales como arquitectura en tres capas, autenticación de usuarios, consumo de API REST y uso de bases de datos relacionales.

Además, permite aplicar buenas prácticas de desarrollo ágil, trabajo en equipo y despliegue en la nube, consolidando conocimientos técnicos en backend, frontend y administración de servidores.

Mundo del problema

Las plataformas digitales permiten a los usuarios organizar su música mediante listas personalizadas y realizar búsquedas rápidas dentro de amplios catálogos musicales. Sin embargo, para comprender el funcionamiento interno de este tipo de sistemas, es necesario modelar una solución que gestione información musical de manera estructurada.

El problema que aborda MusicZone consiste en diseñar e implementar una aplicación web que permita:

- Autenticación segura de acceso.
- Visualización de un catálogo musical.
- Búsqueda eficiente de canciones.
- Creación y gestión de listas de reproducción.

El sistema debe almacenar la información en una base de datos relacional y ofrecer una interfaz web que interactúe con un backend desacoplado mediante una API REST.

Requerimientos Funcionales

- RF1. El sistema debe permitir el inicio de sesión mediante nombre de usuario y contraseña.
- RF2. El sistema debe validar las credenciales contra la base de datos.
- RF3. El sistema debe mostrar un mensaje de error en caso de autenticación fallida.
- RF4. El sistema debe redirigir al usuario autenticado a la pantalla principal.
- RF5. El sistema debe permitir visualizar el catálogo completo de canciones.
- RF6. El sistema debe permitir buscar canciones por título.
- RF7. El sistema debe permitir buscar canciones por artista.
- RF8. El sistema debe permitir crear nuevas listas de reproducción.
- RF9. El sistema debe permitir editar el nombre de una lista.
- RF10. El sistema debe permitir eliminar una lista de reproducción.
- RF11. El sistema debe permitir agregar canciones a una lista.
- RF12. El sistema debe permitir eliminar canciones de una lista.

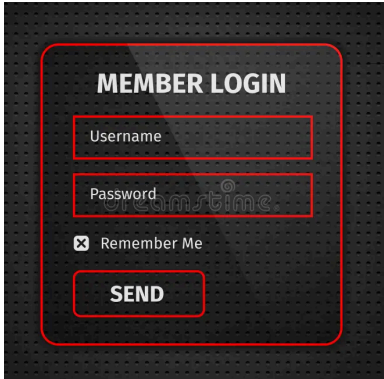
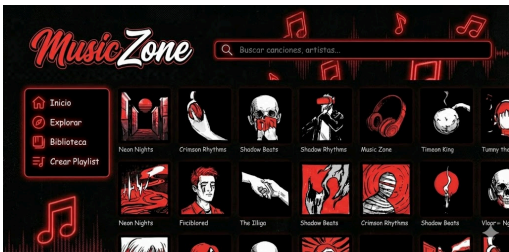
Requerimientos no funcionales

- RNF1. El backend debe estar implementado en Spring Boot y exponer una API REST.
- RNF2. El frontend debe estar desarrollado utilizando HTML, CSS y JavaScript.
- RNF3. La base de datos debe ser relacional.
- RNF4. El sistema debe garantizar autenticación para acceder a las funcionalidades principales.
- RNF5. El sistema debe estar alojado en un servidor en la nube.
- RNF6. El código fuente debe gestionarse mediante un repositorio en GitHub.
- RNF7. El sistema debe contar con ambientes separados de desarrollo y producción.
- RNF8. El tiempo de respuesta en las consultas no debe afectar la experiencia básica del usuario.

Evidencia de las historias de Usuario

ID	Historia de Usuario	Prioridad
HU1	Inicio de sesión	Alta
HU2	Ver catálogo	Media
HU3	Buscar canción	Media
HU4	Crear lista	Alta
HU5	Agregar canciones	Baja

Interfaz Gráfica

Requerimiento	Propuesta
1. Usuario y contraseña. 	
2. Pantalla de ingreso. 	
3. Modulo de Usuarios. 	